

WEBTOON READER PWA

Product Requirements Document

A personal, self-hosted Progressive Web App for aggregating and reading webtoons and manga from multiple configurable sources — inspired by Comick.io.

Version	1.0.0
Project Type	Personal / Self-Hosted
Platform	PWA (Android & iOS)
Date	March 2026
Status	Pre-Development

TABLE OF CONTENTS

1	Project Overview
2	System Architecture
3	Technology Stack
4	Application Structure & Navigation
5	Screen-by-Screen Requirements
5.1	Home Page
5.2	Search Page
5.3	Title Detail Page
5.4	Reader Screen
5.5	Library Page
5.6	History Page
5.7	Settings Page
5.8	Add Source Screen
5.9	Stats Page
6	Source Management System
7	Feature Requirements
8	Performance Requirements
9	PWA Requirements
10	Data Management
11	Future Enhancements

What is it?

Webtoon Reader PWA is a personal, self-hosted web application that aggregates webtoons and manga chapters from multiple external websites into a single, unified reading experience. It is inspired by Comick.io and designed to be installed as a native-like app on mobile devices via the Progressive Web App standard.

Goals

- Aggregate content from any website by configuring CSS selectors — no hardcoded scrapers.
- Provide a Comick-quality reading experience: smooth scrolling, progress tracking, bookmarks.
- Work offline for previously read chapters via service worker caching.
- Be fully installable on Android and iOS as a home screen app.
- Store all user data locally — no accounts, no cloud, complete privacy.
- Run on a local Node.js backend on the user's machine, accessible from their phone via Wi-Fi.

Out of Scope

- No user authentication or multi-user support.
- No cloud sync (all data stays on device / local network).
- No content hosting — the app only scrapes and proxies existing public websites.
- No monetisation features.

2 — SYSTEM ARCHITECTURE

The system is split into two parts: a lightweight Node.js backend that runs on the user's computer and a React PWA frontend installed on their phone. Communication happens over the local Wi-Fi network.

Frontend	React + Vite PWA	UI, reader, bookmarks, offline caching via service worker
Backend API	Node.js + Express	Scraping, caching, image proxy, source management
Scraping Engine	Cheerio + Puppeteer	Parse static HTML sites and JS-rendered sites
Local Storage	IndexedDB	Reading progress, bookmarks, chapter cache, settings
Image Proxy	Express route	Bypass CORS restrictions on source site images
Cache	NodeCache (server)	TTL-based caching of scrape results (5 min default)

Data Flow

1. User opens the PWA on their phone (installed as home screen app).
2. PWA makes API requests to the Node.js backend running on the same Wi-Fi network (e.g. `http://192.168.1.5:3001`).
3. Backend checks its in-memory cache. On miss, it fetches the source website HTML.
4. Cheerio or Puppeteer parses the HTML using the stored CSS selectors for that source.
5. Parsed data is returned to the PWA as JSON and rendered in the UI.
6. Chapter images are loaded through the image proxy endpoint to bypass CORS.
7. Reading progress and bookmarks are written to IndexedDB on the device.

3 — TECHNOLOGY STACK

Area	Technology	Reason
Frontend Framework	React 18 + Vite	Fast HMR, small bundle, great PWA support
Routing	React Router v6	SPA navigation with history management
State Management	Zustand	Lightweight, no boilerplate, persists to IndexedDB
Styling	Tailwind CSS	Utility-first, mobile-first, dark mode built-in
Offline Storage	Dexie.js (IndexedDB)	Typed wrapper, stores progress, bookmarks, cache
Service Worker	Workbox (via vite-plugin-pwa)	Offline caching strategies, background sync
Backend Runtime	Node.js 20+	Non-blocking I/O, ideal for concurrent scraping
Backend Framework	Express 4	Minimal, well-documented REST API
HTML Scraping	Cheerio	jQuery-like API for static HTML parsing
JS-Rendered Sites	Puppeteer	Headless Chromium for sites that need JavaScript
HTTP Client	Axios	Promise-based, supports custom headers and proxying
Server Cache	node-cache	In-memory TTL cache for scrape results
Rate Limiting	express-rate-limit	Prevents hammering source websites
Image Proxy	Custom Express route	Streams images, sets correct headers, caches 24h

4 — APPLICATION STRUCTURE & NAVIGATION

The app uses a persistent bottom navigation bar with 5 tabs. Two additional screens (Title Detail and Reader) are pushed as full-screen overlays on top of the tab layout.

	Icon	Screen	Description
1	Home	Home Page	Discovery feed + continue reading + recent history
2	Search	Search Page	Search across all sources, browse by genre
3	Library	Library Page	Bookmarked titles, organized by shelf
4	History	History Page	Full reading history log
5	Settings	Settings Page	Sources, appearance, data, network config
—	Overlay	Title Detail	Title info + chapter list (pushed from any tab)
—	Overlay	Reader	Fullscreen reader (no nav bar visible)

5.1 Home Page

The primary screen users see on launch. Designed around immediate resumption of reading.

Header	App logo (left) · Notification bell · Profile avatar (right)
Continue Reading	Horizontal scroll row — last 6 read titles, cover + title + progress bar + Ch. X badge. Tapping resumes at exact last position. Hidden if no history.
Recent History	Compact horizontal list of last 5 individual chapters opened, with timestamps. 'See All' opens full History page.
Featured Banner	Auto-scrolling hero carousel (5–7 titles). Full-width cover art, title, genre tags, rating, dot indicators.
Recently Updated	Horizontal scroll — titles with new chapters, 'Ch. X — 2h ago' label per card. 'See All' button.
Popular This Week	Horizontal scroll — trending titles aggregated from all source homepages.
New Titles	Horizontal scroll — newly added titles from sources.
Browse by Genre	Row of tappable genre pill buttons — Action, Romance, Fantasy, Horror, Sci-Fi, etc.
From [Source]	One horizontal section per configured source, 6 titles each. 'See All' per source.

5.2 Search Page

Queries all active sources simultaneously. Three visual states.

Search Bar	Auto-focused on tab open. 300ms debounce. Clear (×) button while typing.
Discover State	Shown before typing: trending search tags, recent searches (removable), popular genre grid with cover art backgrounds.
Suggestions State	Live dropdown while typing: cover thumbnail + title + source badge per suggestion. Last item: 'Search for [query]'.
Results State	Result count header. Filter bar: Source / Status / Sort. 2-column card grid: cover, title, source badge, status tag, latest chapter, rating. Infinite scroll.
No Results State	Illustration + message + suggestions to try different keyword or add a new source.

5.3 Title Detail Page

Pushed as an overlay when any title card is tapped. Contains full metadata and chapter list.

Header Bar	Back arrow · Share icon · 3-dot menu (Report, Open in Browser, Copy Link).
Hero Section	Blurred cover as background. Portrait cover thumbnail left. Right: title, author, status badge (Ongoing/Completed/Hiatus), source badge, star rating + count.
Action Buttons	Add to Library (toggles ■ if bookmarked) · Start / Continue Reading · Download (future placeholder).
Overview Tab	Genre tag pills · Synopsis (collapsed 3 lines, 'Read More' expands) · Related titles horizontal scroll.
Chapters Tab	Chapter count · Search within chapters · Sort toggle (Newest/Oldest) · Chapter list: number, optional title, release date, read/unread styling. Tap opens reader.
Comments Tab	Placeholder for future implementation.

5.4 Reader Screen

Fullscreen immersive reader. Bottom navigation bar is hidden. All UI auto-hides on scroll.

Top Bar	Auto-hides on scroll, reappears on tap. Back button · title + chapter number · settings gear.
Reading Area	Vertical scroll mode (webtoon long-strip, default) or horizontal swipe mode (manga page-by-page). Toggle between modes inside reader.
Image Handling	Lazy loading (3 images ahead). Double-tap to zoom. Pinch to zoom. Spinner per image while fetching. Broken image placeholder.
Bottom Bar	Auto-hides. Prev chapter ← · Progress slider (Page X / Y, draggable) · Next chapter →.
Reader Settings	Slide-up panel: reading direction · background color (Black/Dark/White/Sepia) · brightness slider · image fit mode · zoom lock.
Progress Saving	Scroll position auto-saved to IndexedDB. Chapter marked as read when scrolled past 90%.
Chapter End Screen	Shown after last page: 'Chapter X complete' · large Next Chapter button · Back to Title button · local 1–5 star rating.

5.5 Library Page

All bookmarked titles organized into user-defined shelves.

Header	My Library title · Search icon (filter within library) · Sort icon.
Shelf Tabs	Horizontal scrollable tabs: All · Reading · Plan to Read · Completed · On Hold · Dropped. Each shows count badge.
View Toggle	Grid view (2–3 columns) or List view. Persisted in settings.
Grid Card	Cover art · title name · unread chapter count badge (red bubble, top-right) · source badge.
List Row	Small cover · title name · last read (Ch. X, timestamp) · unread count · source badge · status dot.
Sort Options	Last Read · Last Updated · Title A–Z · Date Added.
Long Press Menu	Move to shelf · Mark all as read · Remove from library.
Empty State	Illustration + 'Your library is empty' + 'Browse titles' button.

5.6 History Page

Full chronological log of every chapter read.

Header	Reading History title · Clear All button (with confirmation dialog).
Date Groups	Entries grouped by: Today · Yesterday · This Week · Earlier.
List Item	Cover thumbnail · title name · 'Read Chapter X' label · timestamp. Tapping resumes that chapter.
Empty State	'No reading history yet' + 'Start reading' button linking to Home.

5.7 Settings Page

Profile Section	Local avatar · editable display name · reading stats summary (chapters read, titles, streak).
Sources	List of sources with enable/disable toggles · Edit and delete per source · 'Add New Source' button.
Appearance	Theme: Dark / Light / Sepia / System Default · Accent color picker (6 presets) · Bottom nav style.
Reader Defaults	Default reading mode · default background color · auto-advance chapters toggle.
Storage & Cache	Cache size display · Clear image cache · Clear chapter cache.

Data Management	Export library + progress as JSON · Import from JSON · Reset all data (with confirmation).
Network	Backend URL input · Test connection button (■ / ■) · Request timeout slider · Image proxy toggle.
About	App version · build info · GitHub link (if published).

5.8 Add Source Screen

A 3-step wizard for configuring a new scraping source.

Step 1 — Basic Info	Source name · Base URL · Render type: Static HTML (Cheerio) or JS-Rendered (Puppeteer).
Step 2 — Selectors	Tabbed form: Search · Title · Chapters · Reader. Each tab has labeled CSS selector inputs. Live preview panel shows scraped results in real time. Green = found, Red = not found.
Step 3 — Test & Save	Full test run across all 4 selector groups. Results preview. 'Save Source' button.

5.9 Stats Page

Summary Cards	Total chapters read · Total titles started · Total titles completed.
Reading Streak	GitHub-style calendar heatmap showing reading activity by day.
Genre Breakdown	Pie chart of most-read genres.
Daily Activity	Bar chart of chapters read per day for the last 30 days.
Top Titles	Top 5 most-read titles by chapter count.

The most technically unique part of the app. Instead of hardcoded scrapers, each source is fully configurable via CSS selectors stored in a local JSON file on the backend.

How Sources Work

- Each source has a unique ID, name, base URL, render type, and a set of CSS selectors.
- Selectors are organized into 4 groups: Search, Title Page, Chapter List, and Reader Pages.
- The scraper uses whichever selector group is needed for the current API endpoint.
- Cheerio handles static HTML. Puppeteer is used when JavaScript rendering is required.
- Sources can be enabled or disabled without deletion.
- A per-source scrape delay can be configured to avoid rate limiting.

Selector Groups

	Selectors Defined
Search	urlTemplate, item container, title text, title link, cover image, coverAttr, latestChapter, rating
Title Page	title name, cover image, coverAttr, description, genre tags, status, author
Chapter List	chapter item container, chapter number, optional chapter title, link href, release date
Reader Pages	image element selector, image attribute (src / data-src / data-lazy)

Reading Experience

- Vertical scroll mode (webtoon long-strip) — default
- Horizontal swipe mode (manga page-by-page)
- Toggle reading mode inside the reader without leaving
- Auto-save scroll position per chapter to IndexedDB
- Mark chapter as read when 90% scrolled
- Double-tap and pinch-to-zoom
- Auto-hide top and bottom bars on scroll
- Full-screen mode (hide browser chrome)
- Chapter end screen with next chapter prompt
- Reader appearance settings: background, brightness, image fit
- Previous / Next chapter navigation
- Draggable page progress slider

Library & Bookmarks

- Add / remove any title to personal library
- Organize titles into 5 shelves: Reading, Plan to Read, Completed, On Hold, Dropped
- Show unread chapter count badge on library cards
- Sort library by: last read, last updated, title A–Z, date added
- Filter library by shelf
- Grid and list view modes
- Long-press context menu on library cards

Progress Tracking

- Per-title: last chapter read, last page position, timestamp
- Per-chapter: read / unread status
- Full history page with date-grouped entries
- Local 1–5 star rating per chapter (stored in IndexedDB)
- Continue Reading strip on Home showing 6 in-progress titles
- Recent History strip on Home showing last 5 chapters

Search & Discovery

- Global search across all active sources simultaneously
- Filter results by source, status, and sort order
- Browse by genre page
- Source-specific homepage sections on Home

- Live suggestions dropdown while typing
- Recently Updated and Popular This Week feeds
- Trending searches and recent search history

8 — PERFORMANCE REQUIREMENTS

	Target	Implementation
Image Loading	Lazy — only load visible + 3 ahead	Intersection Observer API
Search Debounce	300ms delay before firing requests	useDebounce hook
API Cache TTL	5 minutes for most endpoints, 2 min for recent	node-cache on backend
Image Proxy Cache	24 hours server-side	Cache-Control header in Express
Offline Support	Previously read chapters available offline	IndexedDB + Service Worker
Skeleton Loaders	All loading states show skeleton UI, no blank screens	React skeleton components
Chapter Prefetch	Next chapter images start loading at 80% scroll	Background fetch on scroll event
Bundle Size	< 300KB initial JS bundle	Code splitting via React.lazy
Time to Interactive	< 2s on 4G connection	Vite optimised build + caching

9 — PWA REQUIREMENTS

The app must meet the full PWA criteria to be installable on both Android and iOS as a native-like home screen application.

- Web App Manifest (manifest.json) with app name, short name, icons (192px + 512px), theme color, background color, and display: standalone.
- Service Worker registered via Workbox (vite-plugin-pwa) — handles install, activate, and fetch events.
- HTTPS required for production (or localhost for development).
- Installable on Android Chrome and iOS Safari (Add to Home Screen).
- Works in both landscape and portrait orientation.
- Standalone display mode — browser UI chrome hidden when launched from home screen.
- Offline fallback page shown when network is unavailable and no cache exists.
- Background sync for reading progress (future — requires Push API permission).
- App icon and splash screen configured per platform.
- Meta viewport tag: width=device-width, initial-scale=1, viewport-fit=cover (for notched phones).

IndexedDB Schema (Client)

	Key	Value
progress	titleId + chapterId	{ page, scrollPct, readAt, completed }
bookmarks	titleId	{ title, cover, shelf, addedAt, sourceId }
history	timestamp (auto)	{ titleId, chapterId, chapterNum, readAt }
chapterCache	chapterId	{ pages: [...imageURLs], cachedAt }
settings	key string	Any settings value (theme, backend URL, etc.)
ratings	chapterId	{ stars: 1-5, ratedAt }
sources_local	sourceId	Local status overrides (enabled/disabled)

Backend Storage (Server)

- sources.json — array of source configs with selectors, stored as a local JSON file.
- NodeCache — in-memory cache for scrape results, cleared on server restart.
- No database required — the backend is stateless beyond sources.json.

Export / Import Format

Users can export all their data as a single .json file containing:

- All bookmarks and shelf assignments
- All reading progress records
- Full reading history
- Local ratings
- Settings preferences

The same file can be re-imported to restore state on a new device or after a reset.

11 — FUTURE ENHANCEMENTS

These features are out of scope for v1.0 but are planned for future versions.

Push Notifications

Notify user when a bookmarked title gets a new chapter (requires background sync + notification permission).

MAL / AniList Sync

Connect to MyAnimeList or AniList to sync reading progress with the user's external lists.

Bulk Mark as Read

Select multiple chapters at once and mark them all as read in one action.

Chapter Pre-fetching

Auto-download the next chapter in the background while the user is reading the current one.

Custom CSS Per Source

Inject custom CSS into Puppeteer sessions to dismiss cookie banners, paywalls, and overlays.

Reading History Timeline

Visual calendar timeline of reading activity with title covers.

Tablet / Desktop Layout

Two-panel layout for tablets: chapter list on left, reader on right.

Multiple Profiles

Support multiple user profiles (shelves, progress, history) on the same device.

Manga OCR

Optional on-device OCR for raw manga pages to enable text selection and translation.